



# Top 50 Git & GitHub Interview Questions

By Devops Shack

---

[Click here for DevSecOps & Cloud DevOps Course](#)

## DevOps Shack

# Top 50 Git & GitHub Interview Questions

### ☒ 1. What is a branch in Git?

**Answer:**

A branch in Git is a separate line of development used to isolate work. You can experiment or build features without affecting the main code. For example, creating a branch `feature/login` allows working on login functionality without touching `main`. When the feature is tested, the branch can be merged back to `main`.

### ☒ 2. Why do companies use branching strategies?

**Answer:**

Branching strategies help teams work in parallel without breaking production code. They ensure stability, code reviews, CI pipelines, and gradual feature rollout. Examples: GitFlow, Trunk-Based Development, GitHub Flow. They reduce conflicts, improve code quality, and help release planning.

### ☒ 3. How do you create and switch to a new branch?

**Answer:**

`git checkout -b feature/payment` creates and switches to a branch called `feature/payment`.

It does two things at once: (1) creates a branch from the current HEAD and (2) moves the working directory into that branch.

### ☒ 4. What is the difference between `git checkout` and `git switch`?

**Answer:**

`git checkout` is older and used for switching branches, reverting files, and more.

`git switch` is newer and specifically for switching branches.

Example: `git switch feature/api` is safer because it cannot accidentally discard file changes like `git checkout` could.

#### ☒ 5. How do you list all branches?

**Answer:**

`git branch` shows local branches.

`git branch -a` shows local + remote branches.

Useful for identifying which branches to merge or delete.

#### ☒ 6. How do you delete a branch locally and remotely?

**Answer:**

Local: `git branch -d feature/cart` (safe delete) or `-D` (force).

Remote: `git push origin --delete feature/cart`.

Deleting stale branches keeps repositories clean and avoids confusion.

#### ☒ 7. What is merging in Git?

**Answer:**

Merging means combining changes from one branch into another. Git compares commit histories and integrates the changes. Example: merging `feature/profile` into `main` adds the feature code into production.

#### ☒ 8. Difference between fast-forward merge and three-way merge?

**Answer:**

Fast-forward: `main` moves forward directly to feature branch if no diverging commits exist.

Three-way merge: `main` and `feature` have different commits, so Git creates a merge commit combining histories.

Fast-forward produces a cleaner history; 3-way merge keeps track of merge points.

☒ 9. Why do some teams disable fast-forward merge?

**Answer:**

Because fast-forward hides merge history.

Teams prefer merge commits to show exactly when a feature was integrated. This improves traceability and rollback.

☒ 10. How do you perform a merge?

**Answer:**

Switch to the target branch → `git merge feature/api`.

It merges feature/api into the current branch. If conflicts occur, they must be resolved manually before completing the merge.

☒ 11. What causes merge conflicts?

**Answer:**

Conflicts occur when two branches modify the same code part differently. Git cannot decide whose change is correct, so manual resolution is needed. Example: two developers edit the same function logic in different ways.

☒ 12. How do you resolve merge conflicts?

**Answer:**

Open conflicting files, review <<<<<< HEAD, =====, >>>>>> feature/login markers, choose or combine the correct logic, remove markers, save file, then `git add .` → `git commit`. Tools like VS Code, IntelliJ, P4Merge help visualize conflicts.

☒ 13. What is the role of `git merge --abort`?

**Answer:**

It cancels an ongoing merge and returns the repository to the previous state before the merge started. Useful when the merge is too complex or accidental.

---

☒ 14. What is `git merge --no-ff`?

**Answer:**

It forces a merge commit even if a fast-forward merge is possible. This preserves feature history for audit and release tracking.

☒ 15. What is `git cherry-pick`?

**Answer:**

Cherry-pick applies a specific commit from one branch to another instead of merging the full branch.

Example: if a bug fix exists in `dev` and must reach `main` quickly, you cherry-pick that commit without merging all dev changes.

☒ 16. When is `cherry-pick` preferred over merge?

**Answer:**

When you only need one or few specific commits from another branch and don't want all other commits or unfinished features. Cherry-pick is common during hotfixes.

☒ 17. How do you cherry-pick a commit?

**Answer:**

Identify commit hash → `git cherry-pick a1b2c3d`.

This applies that commit to the current branch, possibly creating conflicts if code has changed in between.

☒ 18. Can `cherry-pick` cause conflicts?

**Answer:**

Yes. If the selected commit modifies code that has changed in the target branch, Git cannot determine merging logic automatically. Manual conflict resolution is needed.

---

☒ 19. Difference between merge and cherry-pick?

**Answer:**

Merge brings all commits from the branch.

Cherry-pick brings only selected commits.

Merge keeps history intact; cherry-pick duplicates commit history.

☒ 20. What is a Git revert?

**Answer:**

Revert undoes the effects of a commit by creating a new commit that reverses the previous changes. It does not remove commit history and is safe for public branches.

☒ 21. How is revert different from reset?

**Answer:**

Reset removes commits (rewrites history) and is unsafe on shared branches.

Revert creates new commits that undo previous commits, keeping history intact and safe for remote branches.

☒ 22. How do you revert a commit?

**Answer:**

Find hash → `git revert a1b2c3d`.

If conflicts appear during the revert, resolve them, add, and commit.

☒ 23. How do you revert a merge commit?

**Answer:**

Use the parent flag:

```
git revert -m 1 <merge_commit_hash>
```

This tells Git which parent branch to treat as the base.

☒ 24. Can revert be undone?

**Answer:**

Yes. Revert creates a new commit, so revert can be reverted again using another `git revert` or by cherry-picking the original commit.

☒ 25. Why do companies use Pull Requests (PR) or Merge Requests (MR)?

**Answer:**

For code reviews, CI testing, static scanning, comments, approvals, and controlled merging. PRs prevent accidental direct pushes to production branches.

☒ 26. What is GitHub Fork vs Branch?

**Answer:**

Branch exists inside the same repository — for team collaboration.

Fork creates a full copy of the repository — for open source contributors who don't have write access.

☒ 27. What is the HEAD pointer?

**Answer:**

HEAD points to the current checkout location. It represents the latest commit of the active branch.

Example: switching branches moves the HEAD to a different branch tip.

☒ 28. How do you check commit history of a branch?

**Answer:**

`git log` or `git log --oneline --graph --decorate` for a visual tree format. It helps identify where merges or cherry-picks happened.

☒ 29. What is `git diff` used for?

**Answer:**

It compares changes between commits, branches, staged vs unstaged files, etc.

Example: `git diff main...staging` shows differences between branches before merging.

☒ 30. How do you cancel uncommitted local changes?

**Answer:**

`git restore .` removes unstaged changes.

`git checkout -- filename` is older method.

Useful when wrong modifications were made unintentionally.

☒ 31. What is a protected branch in GitHub?

**Answer:**

A branch with restricted push access — e.g., `main` or `production`. It requires PR approvals, status checks (CI), or security scans before merging.

☒ 32. What is rebasing and why is it not asked here instead of merging?

**Answer:**

Rebasing rewrites commit history. For this question set you requested merging, cherry-pick, revert, and conflict resolution — but rebasing is related.

Rebase = linear history; Merge = keeps branch history.

☒ 33. How do you identify the commit that introduced a bug?

**Answer:**

`git blame` shows line-by-line attribution.

`git bisect` helps binary search among commits to detect the faulty one.

☒ 34. What is a detached HEAD state?

**Answer:**

It happens when checking out a commit directly instead of a branch.  
Commits made here may be lost unless a branch or tag is created.

☒ 35. What is squashing commits?

**Answer:**

Combining multiple commits into one. Teams squash commits during PR merge to keep history clean and prevent 50 micro-commits from cluttering main.

☒ 36. When should teams delete feature branches?

**Answer:**

After merging to main and confirming deployment. Deleting prevents clutter and stops accidental reuse of outdated branches.

☒ 37. What is `git stash` during conflict resolution?

**Answer:**

If you want to pull changes but your local changes interfere, `git stash` temporarily stores them. Later, `git stash apply` retrieves the changes for testing.

☒ 38. How to resolve conflict during cherry-pick?

**Answer:**

Follow same process as merge conflicts: open conflict markers, adjust logic, save, `git add`, `git cherry-pick --continue`.

☒ 39. What are conflict markers in Git?

**Answer:**

Example:

```
<<<<<<< HEAD
your change
=====
incoming change
>>>>>>> feature/login
```

Everything between <<<<<<< and ===== is local version; everything between ===== and >>>>>>> is incoming version.

☒ **40. What is the purpose of `.gitattributes` in conflict resolution?**

**Answer:**

You can define custom merge strategies for certain files. Example: always take current branch for \*.lock files.

☒ **41. What is `git restore --staged` file used for?**

**Answer:**

It removes the file from staging area without removing your content. Useful during conflict resolution when a mistake is staged accidentally.

☒ **42. What is the difference between `git pull` and `git fetch` when resolving conflicts?**

**Answer:**

`git fetch` downloads changes only.

`git pull` downloads + merges.

During conflict situations, fetching first gives better control.

☒ **43. How to undo a mistaken merge before pushing to remote?**

**Answer:**

---

`git reset --hard HEAD~1` removes the merge commit while local. Do not use on shared branches after push.

☒ **44. How to undo a merge after pushing to remote?**

**Answer:**

Use `git revert <merge_commit_hash>` because revert is safe for shared history and does not delete commits.

☒ **45. Why is merge conflict resolution a key interview topic?**

**Answer:**

Because conflicts occur daily in real projects. Candidates are expected to understand tools, thought process, communication during conflict, and how to avoid conflicts via branching strategy.

☒ **46. How to avoid conflicts during merges?**

**Answer:**

Pull changes frequently, keep feature branches small, avoid editing same files unnecessarily, communicate with team, add CI gates, enforce code ownership on critical modules.

☒ **47. Why should testing be done after merging?**

**Answer:**

Because two individually working features may collide when combined. Integration bugs only appear after merge, so unit + integration + pipelines must run post-merge.

☒ **48. What is conflict during revert?**

**Answer:**

If the reverted commit touches code that has since changed, Git cannot auto-undo changes

---

and requires manually adjusting logic. Similar to merge conflict.

☒ **49. What is the golden rule of public branches?**

**Answer:**

Never rewrite history (via reset or rebase). Instead use revert, merge, cherry-pick, and PR-based approval to ensure traceable history.

☒ **50. What is the biggest real-world takeaway from branching, merging, cherry-pick, revert, and conflict resolution?**

**Answer:**

Use branches for isolated work, merge through PR for transparency and CI checks, cherry-pick only when selective commits are needed, revert mistakes safely without history loss, and resolve conflicts thoughtfully by understanding code intent. Good branching discipline reduces conflicts and accelerates delivery.